

# Base vs SFT vs GRPO on Math Benchmarks

## Goal

这份文档要回答三个问题：

1. 当前 Base / SFT / GRPO 在两个数学 benchmark 上的结果分别是什么？
2. 为什么会出现稳定的 Base > SFT > GRPO 排序？
3. 这个结论有多少来自真实能力变化，又有多少受到 evaluation 方式影响？

## Benchmark Results First

先看结果。当前两个 benchmark 都给出了同样的排序：Base > SFT > GRPO。

### 1. Standard benchmark: `lm-eval minerva_math`

配置：

- 4-shot
- limit=50 per subtask
- 7 个 subtasks, 共 350 题 / 模型
- greedy decoding
- max\_gen\_toks=1024

#### Model exact\_match math\_verify

|      |       |        |
|------|-------|--------|
| Base | 9.43% | 13.43% |
| SFT  | 0.00% | 8.29%  |
| GRPO | 0.00% | 4.86%  |

这说明：

- Base 在标准化 minerva\_math 上最好
- SFT 仍有一定 math\_verify, 说明不是完全不会做
- GRPO 比 SFT 更差, 而且差距不只体现在格式敏感的 exact\_match, 也体现在更宽松的 math\_verify

### 2. Supplementary benchmark: GSM8K xml + 手写脚本

配置：

- limit=100
- XML-style prompting
- 自定义答案提取与错误归因

### Model accuracy strict\_accuracy content\_accuracy

|      |        |       |        |
|------|--------|-------|--------|
| Base | 10.00% | 0.00% | 25.00% |
| SFT  | 6.00%  | 0.00% | 18.00% |
| GRPO | 1.00%  | 0.00% | 15.00% |

这说明：

- 即使换成另一套评测，排序仍然是 Base > SFT > GRPO
- 即使看更宽松的 content\_accuracy，SFT 和 GRPO 也没有反超 Base

## 3. Combined takeaway

两个 benchmark 的共同信息是：

1. 当前观察到的退化趋势不是单一 benchmark 的偶然现象
2. SFT < Base 和 GRPO < SFT 都有跨评测一致性
3. 但两个 benchmark 的含义并不完全一样：
  - minerva\_math 更适合做主 benchmark
  - GSM8K xml + 手写脚本 更适合做行为诊断

## Benchmark Roles and Setup

本项目里两个 benchmark 的定位应该分开理解：

### 1. minerva\_math

这是主 benchmark，因为它更标准、更接近 community 的数学评测口径。

当前配置是：

- benchmark: lm-eval minerva\_math
- generation: generate\_until
- few-shot: 4-shot
- decoding: greedy (do\_sample=false, temperature=0)
- length: max\_gen\_toks=1024
- scope: limit=50 per subtask

### 2. GSM8K xml + 手写脚本

这是补充 benchmark，因为它更适合解释：

- truncation
- format drift
- extraction failure
- “会做但没落好 final answer”

但它不适合单独作为主数学分数，因为其 accuracy 带有 last\_number fallback，并且混合了：

- 数学能力
- XML 格式遵循
- 截断
- 自定义提取策略

## What minerva\_math Is Actually Measuring

当前 minerva\_math 的判分方式需要分开看：

### 1. exact\_match

minerva\_math 的 exact\_match 不是“看到正确数字就算对”，而是先从生成结果里提取：

Final Answer: The final answer is ... I hope it is correct.

然后再做规范化和等价判断。

这意味着：

- 是否能稳定收束到 benchmark 偏好的 final-answer 模板，非常重要
- 长解释、跑偏、没收尾、收尾格式变形，都会直接伤害 exact\_match

### 2. math\_verify

math\_verify 更宽松一些。它会把整段 candidate output 送进数学 verifier，与 gold solution 做符号/数学一致性判断。

所以：

- 它比 exact\_match 更能容忍格式差异
- 但它仍然不是“只要 somewhere 出现了正确数字就算对”
- 如果模型在后半段严重跑偏、算错、输出垃圾 token / HTML / 残缺 XML，math\_verify 也会失败

### 3. 为什么 minerva\_math 仍然应该做主结果

虽然 minerva\_math 也会受到输出风格影响，但它仍然比自定义 GSM8K xml 更标准，因此更适合作为主 benchmark。

## Why Base Beats SFT

我认为 Base > SFT 主要是输出分布被 SFT 推离了 minerva\_math 偏好的答题方式，而不是简单地说“SFT 完全不会数学了”。

## 1. SFT 数据天然偏长解释、偏 tutor-style

GenAI\_final\_project\_data/outputs/phase\_a\_deepmath/phase\_a\_manifest.json 明确要求：

- require\_any\_r1\_solution: true
- min\_r1\_solution\_chars: 200

这说明训练数据在构造阶段就偏向：

- 长推理
- 长文本解释
- 较少的短、硬、benchmark-friendly final answer

也就是说，这版 SFT 更像是在训练“会讲题的老师”，而不是“稳定输出标准答案的 benchmark solver”。

## 2. SFT 对小模型的分布改写比较激进

本地 SFT 配置显示：

- lora\_target: all
- learning\_rate: 2e-4
- num\_train\_epochs: 3.0
- cutoff\_len: 1024 / 1536

对 Qwen2.5-0.5B 这种小模型来说，这种设置很容易把 base 原本更短、更贴 few-shot 模板的输出分布推走。

## 3. Base 更贴近 Minerva few-shot exemplar 的收尾风格

minerva\_math 自带的 few-shot 样例是这种风格：

- Solution: ...
- $\boxed{\dots}$
- Final Answer: The final answer is ... I hope it is correct.

而 Base 在实际样本里，恰好更接近这个风格。

例如在“vertical asymptotes”这题上，Base 直接给出：

- denominator factorization 正确
- $\boxed{2}$
- Final Answer: The final answer is 2. I hope it is correct.

这是非常 benchmark-friendly 的输出。

## 4. SFT 的问题更像“会做，但落不到干净 final answer”

SFT 的样本里经常出现这种模式：

- 开头在做对的方向
- 中间继续自我解释
- 后面开始重复、串台、或延伸到无关内容
- 最终没有稳定地落到干净 final answer

例如同样那道“vertical asymptotes”题，SFT 前半段先正确写出：

- $x^2 + x - 6 = 0$
- $(x + 3)(x - 2) = 0$
- $x = -3, 2$

但后面却突然跳到另一道 few-shot 样例里的系统方程：

- Original equation:  $6x - 4y = a$ ,  $6y - 9x = b$

然后开始长段重复。

这说明 SFT 的问题不只是 final answer 格式不稳，还包括：

- 长输出中的 coherence 变差
- prompt contamination
- 过度续写导致的退化

## 5. 为什么这里说“和 evaluation 有关”，但又不能全怪 evaluation

SFT 相对 Base 的掉分，确实有相当一部分是 evaluation-sensitive 的：

- exact\_match 对 final answer landing 很敏感
- 单次 greedy 生成对“啰嗦、慢收尾”的模型不友好
- 固定 1024 token 预算会放大长输出的坏处

但也不能完全说这是“被错杀”，因为 SFT 的 math\_verify 也从 13.43% 掉到了 8.29%。

所以更准确的结论是：

- SFT < Base 部分是评测方式放大了输出风格问题
- 但也确实伴随了一定真实推理/稳定性下降

## Why GRPO Beats Neither Base nor SFT

GRPO < SFT 的解释和 SFT < Base 不完全一样。

这里我更倾向于认为：除了 evaluation mismatch 之外，GRPO 本身也把模型往不利于 minerva\_math 泛化的方向继续推了一步。

## 1. GRPO reward 明显偏向 XML / boxed / brevity / local answer heuristics

grpo\_trl/train\_grpo\_trl.py 里实际用了这些 reward:

- format\_progress\_reward\_func
- xml\_tag\_shape\_reward\_func
- strict\_format\_reward\_func
- brevity\_reward\_func
- answer\_similarity\_reward\_func
- answer\_reward\_func
- repetition\_penalty\_func

这说明 GRPO 的优化目标并不是“泛化竞赛数学能力”本身，而是更偏向：

- XML 结构
- boxed answer
- 输出短一些
- 与 ground truth answer 更相似
- 少重复

这套目标对你自己的 GSM8K xml 评测可能有帮助，但和 minerva\_math 的 few-shot plain-solution 风格并不完全一致。

## 2. GRPO 训练分布和 Minerva 测试分布存在明显错位

训练报告里，GRPO 的目标输出格式是：

```
<think>
...
</think>
<answer>
\boxed{final answer}
</answer>
```

而 minerva\_math 的 few-shot prompt 给模型看的示范，却是：

- 普通 Solution:
- 普通 Final Answer: The final answer is ... I hope it is correct.
- 没有 XML

这意味着 GRPO 在测试时面对的是一个和训练 reward 不完全同构的协议。

换句话说，GRPO 学到的“好行为”，并不等于 minerva\_math 需要的“好行为”。

## 3. GRPO 的掉分不只是 exact\_match，而是连 math\_verify 也继续下降

如果 GRPO 只是“格式和 benchmark 不对口”，那么通常会看到：

- exact\_match 很差

- 但 `math_verify` 至少能维持住

但现在实际看到的是：

- SFT `math_verify` = 8.29%
- GRPO `math_verify` = 4.86%

这说明 GRPO 的问题已经不只是收尾格式，而是连数学内容本身的可验证正确性都进一步下降了。

## 4. GRPO 样本里出现了比 SFT 更硬的错误

从 GRPO 的 `minerva_math` 样本看，问题不再只是“长”和“啰嗦”，而是更像：

- 局部算术被破坏
- 提前坏掉
- 夹杂残缺标签/HTML
- 中途截断

例如：

1. 在“vertical asymptotes”题上，GRPO 一开始就把因式分解解释错成：
  - “3 and 12 multiply to 36 and add up to 1”
  - 后面直接喷出一长串 `</div>`
2. 在“120% of 30 vs 130% of 20”题上，GRPO 算出：
  - `120% of 30 = 360`
  - `130% of 20 = 206`
  - 然后输出在 `</math>` 处截断

这类错误比 SFT 的“会做但落点不稳”更严重，因为它已经触及：

- 基础算术
- 局部一致性
- 生成稳定性

## 5. GRPO 更像是“对本地 reward 过拟合”，而不是“对标准 benchmark 泛化更强”

训练报告本身也给出一个信号：

- `strict_format_reward_func/mean` = 0
- `eval_completions/clipped_ratio` = 0.8828

这说明即使在训练/验证协议内部，模型的 clean XML stopping 行为也没有真正学稳。

因此，一个比较合理的解释是：

1. GRPO 把模型继续推向 reward-friendly 行为；
2. 但 reward 设计更贴近本地 XML + answer extraction 任务，而不是 `minerva_math`；
3. 结果就是对标准 benchmark 的泛化继续下降。

# How Much Is Due to Evaluation Method

这是当前最需要写清楚的地方。

## 结论先行

当前结果确实与 evaluation 方式有关，但不能完全归因于 evaluation。

## 和 evaluation 相关的部分

当前 minerva\_math 会放大以下问题：

- 长输出
- 晚收尾
- final answer 模板不稳定
- 截断
- 单次 greedy 生成的不稳定性

因此：

- SFT 这种 tutor-style 模型，容易被 exact\_match 低估
- GRPO 如果更依赖 sampling 或更依赖 XML 结构，也可能被 plain-solution few-shot 协议低估

## 不能全怪 evaluation 的部分

但有两个事实不能忽略：

1. 三个模型在更宽松的 math\_verify 上仍然是  
Base (13.43) > SFT (8.29) > GRPO (4.86)
2. SFT / GRPO 样本里都能看到真实的内容层错误，不只是格式问题

所以目前最稳妥的表述应该是：

- SFT < Base：风格/收尾问题 + 一定真实能力/稳定性下降
- GRPO < SFT：objective mismatch + reward over-optimization + 更明显的真实内容退化

# Cross-check with Previous GSM8K Custom Evaluation

更重要的是，之前的 GSM8K + 手写脚本 其实也给出了同样的排序：

| Model | accuracy | strict_accuracy | content_accuracy |
|-------|----------|-----------------|------------------|
| Base  | 10.00%   | 0.00%           | 25.00%           |
| SFT   | 6.00%    | 0.00%           | 18.00%           |
| GRPO  | 1.00%    | 0.00%           | 15.00%           |



这意味着：

- 不是只有 minerva\_math 一套 benchmark 才得到 Base > SFT > GRPO
- 在项目自定义的 GSM8K xml 分析里，也同样得到 Base > SFT > GRPO

这一点很关键，因为它会改变我们对结果的解释力度：

- 如果只有 minerva\_math 一套评测出现这个排序，我们还可以更强地怀疑是 benchmark protocol mismatch
- 但现在两套性质不同的评测都指向同一个方向
- 所以更合理的结论是：当前 Base > SFT > GRPO 不只是单一 benchmark 的偶然产物

## Why This Cross-check Matters

这两套评测的关注点并不一样：

1. minerva\_math
  - 更标准
  - 更适合作为主 benchmark
  - 更接近 community 的数学评测口径
2. GSM8K xml + 手写脚本
  - 更强调生成行为诊断
  - 更容易暴露 truncation、格式漂移、提取失败
  - 不适合作为唯一主分数

当两套不同口径的评测都给出同一个排序时，说明：

- SFT < Base 和 GRPO < SFT 并不只是某一个判分器的 artifact
- 至少在当前项目里的两个观察窗口下，这个退化趋势是稳定存在的

## But GSM8K Does Not Mean the Same Thing as Minerva

这里也要非常谨慎，不能把两者简单等同。

GSM8K xml + 手写脚本 的 accuracy 带有明显的 last\_number fallback；因此它不是纯粹的标准 benchmark 分数。

不过它还有一个很有价值的辅助指标：content\_accuracy。

content\_accuracy 的排序仍然是：

- Base = 25%
- SFT = 18%
- GRPO = 15%

这个指标虽然仍然不是标准 benchmark，但它至少说明：

- 即便放宽到“正确答案有没有出现在输出里”的层面

- 当前也仍然是 Base > SFT > GRPO

所以不能把当前现象完全解释成：

- SFT / GRPO 只是“会做，但因为格式问题被判低”

更准确的说法是：

- SFT / GRPO 的确有格式与收尾问题
- 但即使在更宽松的内容层观察下，它们也没有反超 Base

## Error Pattern Alignment Across Both Evaluations

更进一步看，GSM8K 的错误模式也和 minerva\_math 观察到的现象相互印证。

GSM8K xml 中：

- Base
  - accuracy = 10%
  - content\_accuracy = 25%
  - degeneration 很高 (68%)
- SFT
  - accuracy = 6%
  - content\_accuracy = 18%
  - truncation 很高 (79%)
  - 平均输出更长 (177.5 words)
- GRPO
  - accuracy = 1%
  - content\_accuracy = 15%
  - truncation 更高 (85%)
  - 平均输出最长 (180.6 words)

这和 minerva\_math 样本里看到的趋势是一致的：

- SFT 比 Base 更长、更晚收尾、更容易拖长
- GRPO 比 SFT 更不稳，更容易在长输出中坏掉

因此，两套评测合起来支持的不是一个“彼此矛盾”的故事，而是一个**相互补强**的故事：

1. minerva\_math 说明在标准 benchmark 上，Base > SFT > GRPO
2. GSM8K xml 说明这种下降伴随着：
  - 更长输出
  - 更高 truncation
  - 更差 final-answer landing
3. 所以结果既有“benchmark 不友好”的因素，也有真实生成稳定性下降的因素

## How This Should Be Written in the Report

建议在最终报告里这样解释：

### 1. 主结果

在标准化 `lm-eval minerva_math` 评测下，当前排序为  
Base > SFT > GRPO。

### 2. 对 SFT 的解释

SFT 并非完全失去数学能力；它更像被训练成了长解释、强 tutor-style 的解题器。  
这使它在 `exact_match` 上尤其吃亏，也拖累了部分 `math_verify`。

### 3. 对 GRPO 的解释

GRPO 的 reward 更贴近 XML/boxed/answer extraction 目标，而非 `minerva_math` 的 plain-solution 4-shot 协议。  
结果是它对本地 reward 可能更友好，但对标准 benchmark 泛化更差。

### 4. 对 evaluation 的解释

当前 evaluation 不是“纯粹只测数学内核”，它也会受到输出风格、收尾模板、截断和解码方式影响。  
但由于 `math_verify` 和 `GSM8K content_accuracy` 都同步下降，所以不能把当前排序完全解释成评测偏置。

### 5. 对 supplementary benchmark 的定位

`GSM8K xml` + 自定义评估器 仍然有价值，但更适合作为：

- 格式漂移分析
  - truncation / extraction failure 分析
  - “会做但没落到 final answer” 的诊断工具
- 而不应替代 `minerva_math` 作为主 benchmark。

## Bottom Line

当前最合理的总解释是：

- Base 最贴近 `minerva_math few-shot` 的目标输出分布，因此表现最好
- SFT 被长解释数据和较强 SFT 配置推向了更不 benchmark-friendly 的风格，因此低于 Base
- GRPO 又进一步针对 XML / format / local reward 做了优化，在 `minerva_math` 这种不同协议的标准 benchmark 上继续掉分，因此低于 SFT

所以，现阶段项目报告里应当**如实报告** Base > SFT > GRPO，同时补充说明：

- SFT 的掉分有明显 evaluation-sensitive 成分
- GRPO 的掉分则更像 evaluation mismatch 与真实泛化退化共同作用的结果
- 并且这一排序不只出现在 `minerva_math`，也同样出现在之前的 `GSM8K` + 手写脚本 结果中

## Optimization Suggestions

如果后续要继续优化，我建议把改进分成四层：数据、SFT、RL、evaluation。

## 1. Data: make supervision less “long tutor-only”

当前最明显的问题之一，是训练数据过度偏向长解释、晚收尾的 tutor-style 解答。

建议：

1. 增加更多“短解 + 稳定 final answer”样本
  - 不要只保留长 r1\_solution
  - 显式加入一批短而干净的数学解答样本
  - 目标不是去掉 reasoning，而是让模型同时学会“能展开讲”和“能快速收尾”
2. 为同一道题保留多种 target style
  - 长解释版
  - 短解释版
  - final-answer-first 或 final-answer-clean 版
  - 这样模型不会被单一输出风格锁死
3. 强化 benchmark-friendly final answer supervision
  - 不只是给 `\boxed{final\_answer}`
  - 还要让样本整体更接近真实测试时希望看到的完整收尾格式
  - 例如统一加入简洁、稳定、可解析的 final answer line
4. 对训练数据做“长度与收尾位置”约束
  - 统计 final answer 出现在输出中的相对位置
  - 降低那种“正确答案在很后面，后面还继续讲很多”的样本占比
5. 引入更贴近目标 benchmark 的数据
  - 如果主 benchmark 是 minerva\_math
  - 那么 SFT / RL 数据格式最好至少部分接近 Solution ... Final Answer ... 这种 plain math style
  - 不要让训练分布几乎全是 XML 风格、测试分布却是非 XML

## 2. SFT: avoid over-shifting the base model's output distribution

当前 SFT 看起来不是单纯“增强数学能力”，而是把输出分布明显推向了更长、更不稳定的风格。

建议：

1. 降低 SFT 强度
  - 先尝试更保守的学习率
  - 降少 epoch
  - 缩小 LoRA target 范围
  - 尤其是对 0.5B 小模型，过强 SFT 很容易把 base 原有的好分布一起洗掉
2. 做“mixture-style SFT”
  - 不要只喂长解释数学数据
  - 可以混入一部分 benchmark-friendly math data
  - 或混入一部分短答案/短推理样本，保持输出分布多样性

3. 单独做“收尾能力”微调
  - 专门构造一批数据训练模型在得到答案后及时停止
  - 减少“算对了但继续说，最后跑偏”的情况
4. 增加 generation-side validation
  - 不只看 training/eval loss
  - 每隔固定 step 直接跑小规模生成验证，统计：
    - 平均输出长度
    - final answer 命中率
    - 正确答案首次出现位置
    - 正确后继续输出长度
  - 这样能更早发现“loss 看着正常，但行为变差”
5. 尝试两阶段 SFT
  - 第一阶段保留长解释数据，让模型学 reasoning
  - 第二阶段用较小学习率做 answer-format / concise landing calibration
  - 比一次性用单一长解释目标更稳

### 3. RL / GRPO: align reward with the real evaluation target

当前 GRPO 最大的问题不是“有没有优化”，而是“优化方向和最终 benchmark 不够对齐”。

建议：

1. 重写 reward，使其更接近主 benchmark
  - 如果最终主结果看 minerva\_math
  - 那 reward 就不能主要围绕 XML tag、boxed 个数、brevity 这类局部格式信号
  - 应该更多围绕：
    - 数学等价正确性
    - final answer 稳定性
    - 正确后及时停止
2. 降低格式 reward 的权重
  - 当前 reward 中 XML / strict format / tag shape 占比过重的风险很大
  - 容易把模型推向“看起来像正确格式”，但不一定真的更会做题
3. 提高正确性 reward 的主导性
  - answer correctness / math equivalence 应该是主 reward
  - format 只做辅助约束
  - 否则模型容易 reward hack 到“格式更像，但数学更差”
4. 加入 anti-overthinking / anti-drift reward
  - 如果正确答案已经出现，继续输出过长内容就给轻微惩罚
  - 对重复、自我纠缠、prompt contamination、标签残缺加明确 penalty
5. 让 RL 训练协议更接近测试协议
  - 如果最终要在 minerva\_math 上汇报
  - 可以考虑直接按 plain Problem / Solution / Final Answer 风格构造 RL prompt
  - 而不是只在 XML 协议下训练，再去 plain benchmark 上测试
6. 做更细粒度 ablation
  - 单独去掉 strict\_format\_reward

- 单独去掉 brevity\_reward
- 单独保留 answer\_reward
- 看是哪一项 reward 最伤害 minerva\_math
- 不然很难知道问题是“RL 本身”，还是“某几个 reward 的组合”

#### 7. 优先做短周期 RL 实验

- 先用小步数、频繁 eval 的方式找 reward 方向
- 不要一开始就长跑
- 否则容易在错误 reward 上越训越偏

## 4. Evaluation: make optimization loops more diagnosis-friendly

当前 evaluation 已经足够用来出主结论，但如果要支持下一轮优化，建议再加几类中间指标。

建议：

1. 保留“主 benchmark + 补充诊断”双轨结构
  - 主结果继续用 minerva\_math
  - GSM8K xml + 手写脚本 继续做错误归因
2. 增加 answer landing 指标
  - final answer 首次出现位置
  - 正确答案出现后是否继续输出
  - 正确答案后新增 token 数
3. 增加长度与截断分析
  - 各模型输出长度分布
  - 截断率随 max\_gen\_toks 变化的曲线
  - 这样能判断问题是“不会做”，还是“做出来但太晚”
4. 增加 sample-level paired comparison
  - 同一批题对比 Base / SFT / GRPO
  - 标注：
    - Base 对、SFT 错
    - SFT 对、GRPO 错
    - 三者都错但错法不同
  - 这会比只看 aggregate score 更有解释力
5. 若后续继续做 RL，建议每轮都同时看
  - minerva\_math exact\_match
  - minerva\_math math\_verify
  - GSM8K content\_accuracy
  - 平均输出长度 / truncation rate
  - 这样才能及时看出“reward 上去了，但真实泛化掉了”

## Recommended Practical Next Step

如果只能做一轮最有价值的优化，我建议优先顺序是：

1. 先改 SFT 数据与目标风格

- 混入更短、更 benchmark-friendly 的数学监督
- 保留 reasoning, 但强化干净 final answer landing
- 2. 再做小规模 SFT ablation
  - 更低 learning rate
  - 更少 epoch
  - 更保守 target modules
- 3. 最后再重做 RL
  - 让 reward 以 math correctness 为主, format 为辅
  - 并让训练协议更贴近 minerva\_math

原因是：

- 如果 SFT 初始化已经把模型推偏, 后面的 RL 很可能只会在偏掉的分布上继续优化
- 先把 SFT 这层做好, 再做 RL, 成功概率更高